Supplementary Information

Head-direction invariance, not information, determines whether a predictive cognitive map folds under symmetry

Manas Venkata Sai Ravulapalli

This document collects supporting figures for the main text. All panels are generated directly from the result files by `project5_symmetry/analysis/make_paper_figures.py` and `rate_maps.py`; no values are entered by hand.

1 Place-field rate maps, all extracted units

The main-text place-cell figure shows a small number of example units per condition. To show that this selection is representative rather than chosen, the following figures give the full set of extracted units for each condition, ranked by Skaggs spatial information and left otherwise unselected. Each tile is one unit's mean firing-rate map over the arena, smoothed for display and normalised to its own peak.

C_1, full HD: 40 most spatially informative units

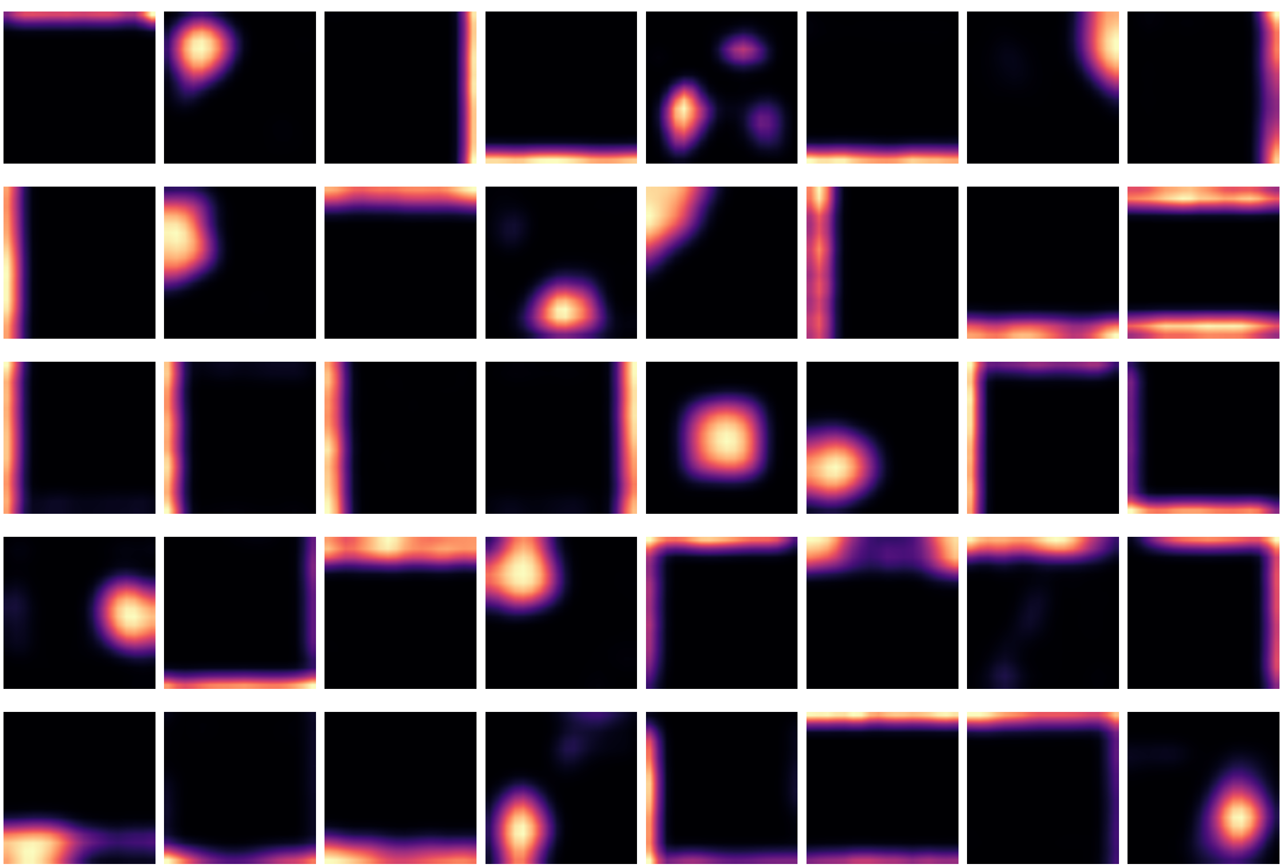

Figure S1. Asymmetric arena, full head-direction encoding. Units carry single, sharp place fields.

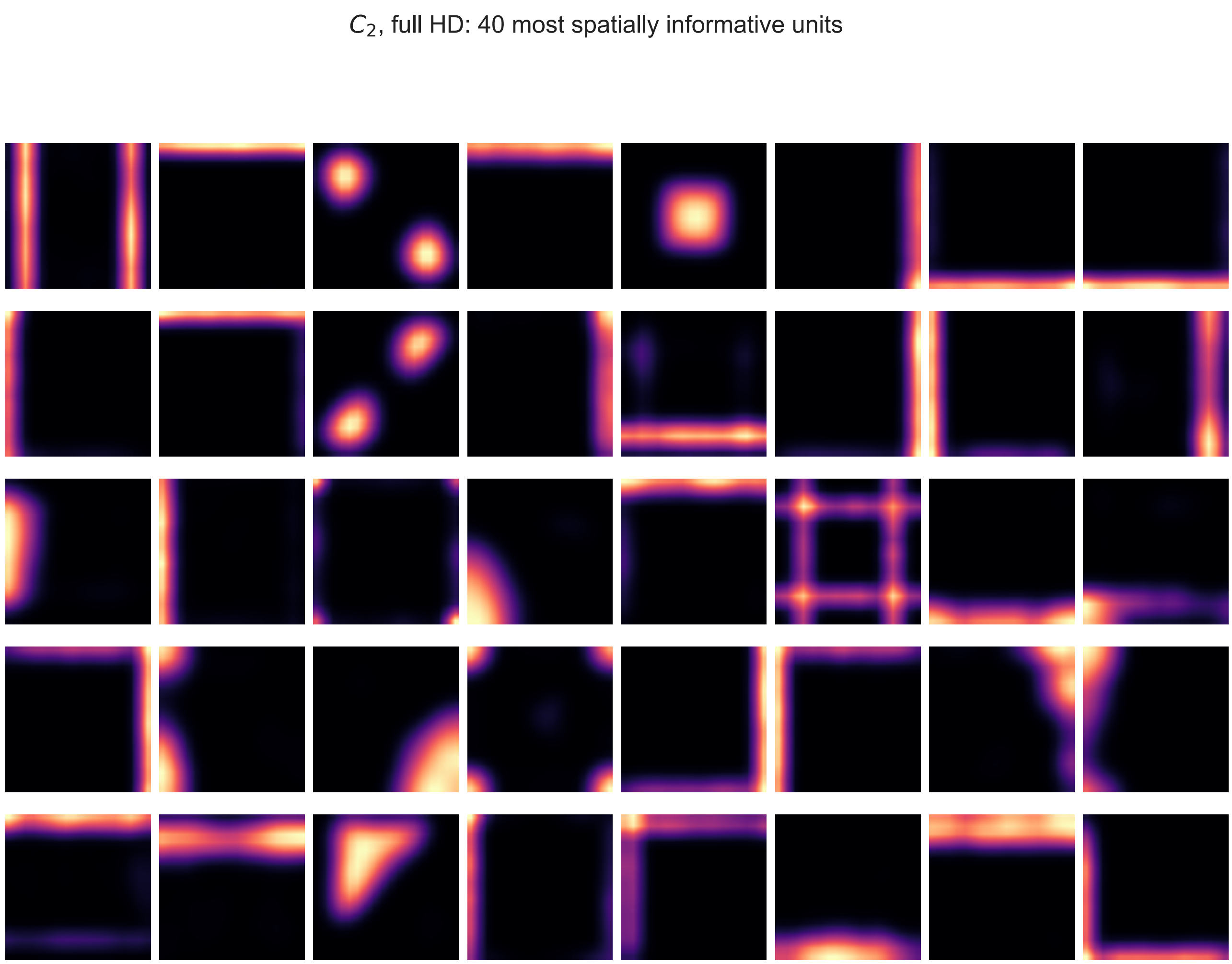

Figure S2. C_2 arena, full head-direction encoding. With the full compass available the fields do not systematically repeat.

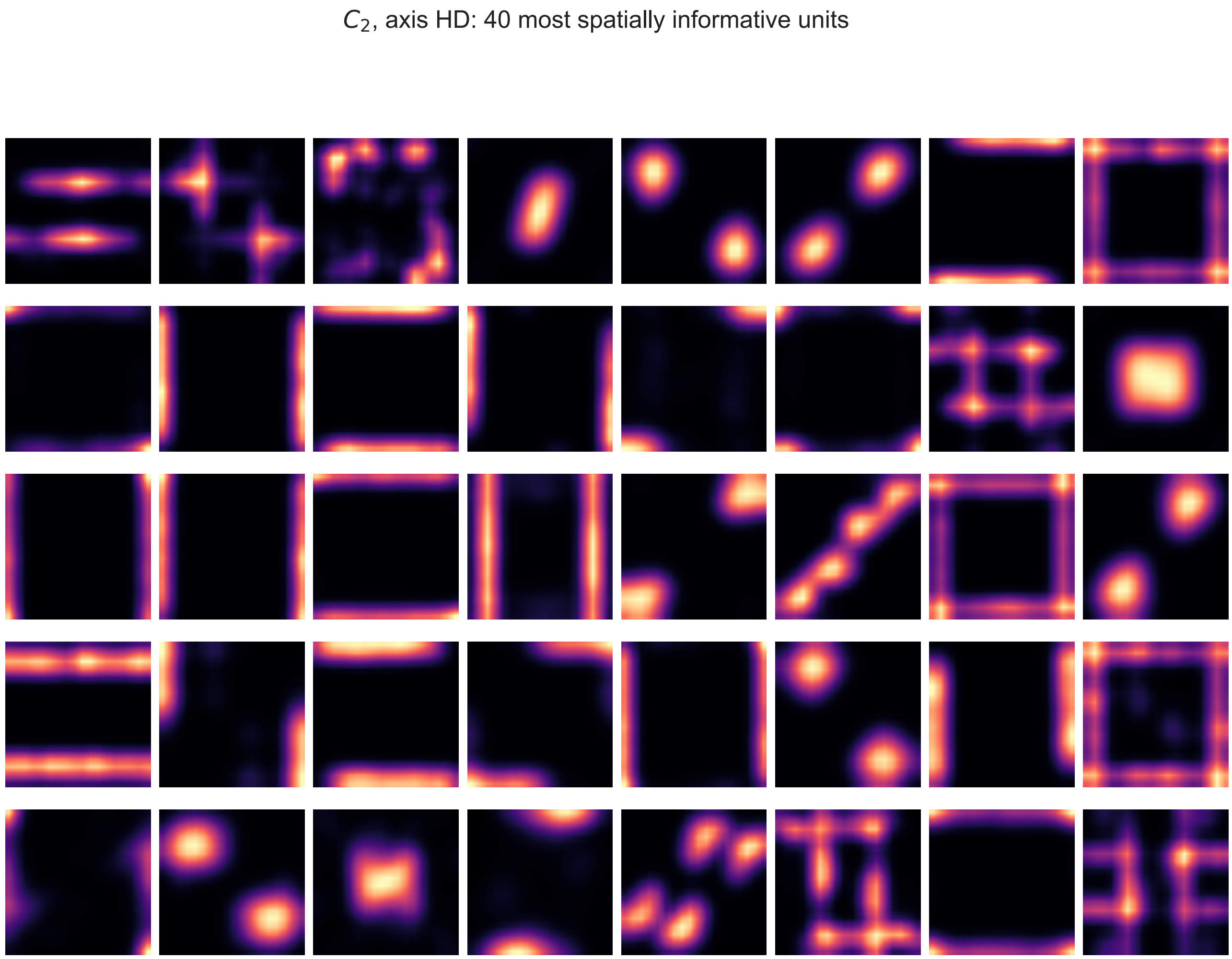

Figure S3. C_2 arena, *axis* encoding. The encoding is invariant under the 180° rotation, and fields repeat at 180°-related locations across the population.

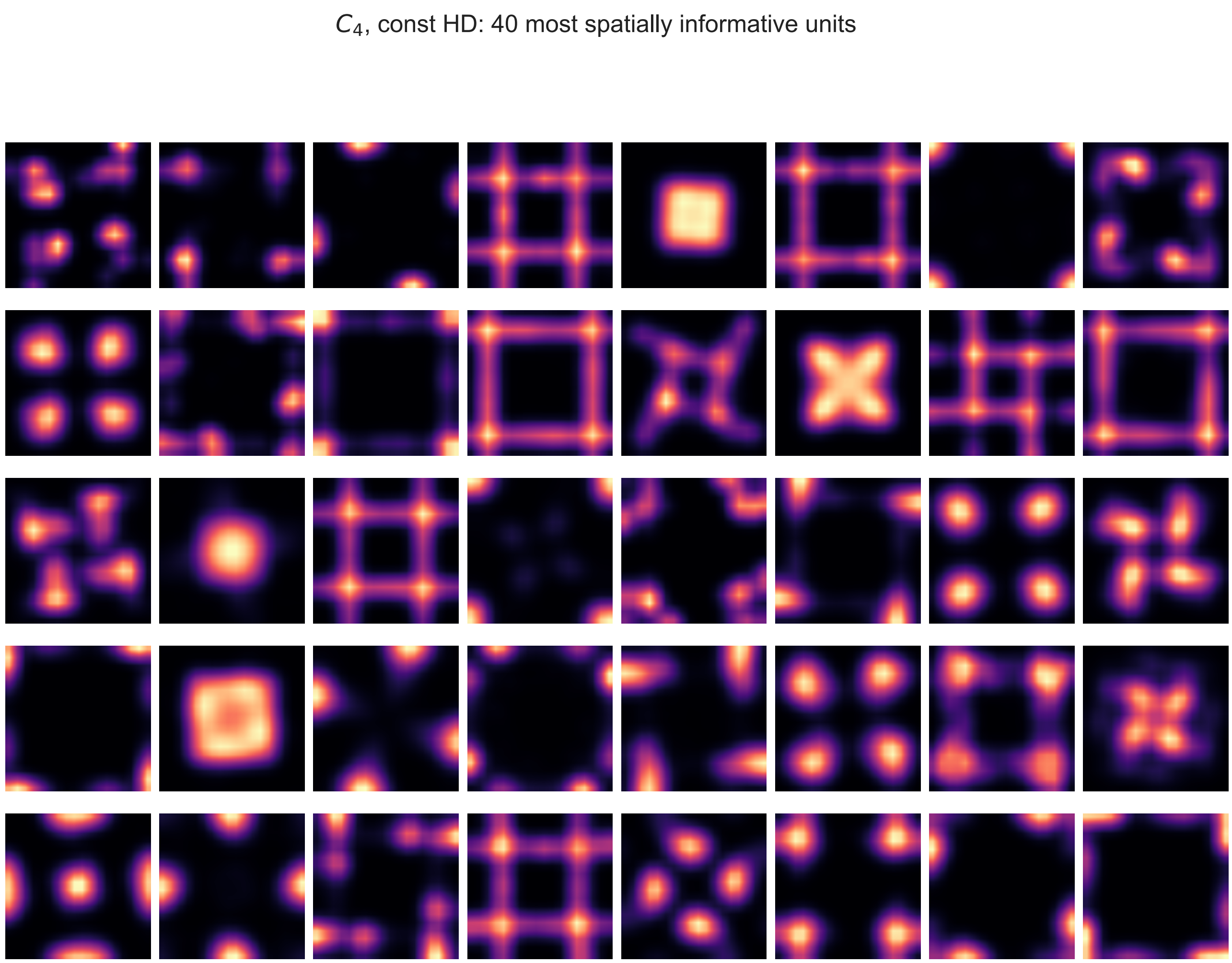

Figure S4. C_4 arena, *const* encoding. The encoding is invariant under the full C_4 group, and fields repeat at all four 90°-related locations.

2 Cell-type composition

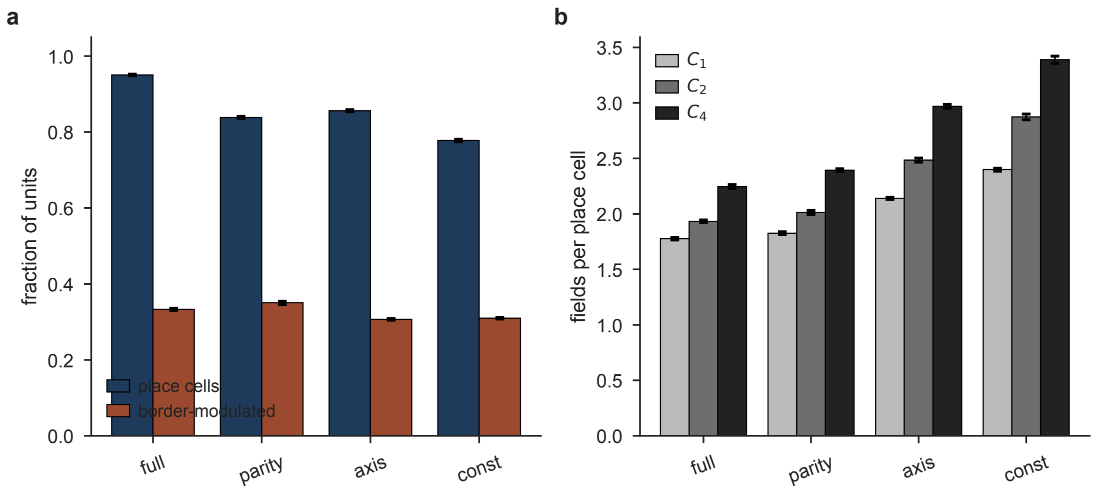

Figure S5. The trained population is a realistic cell-type mixture, scored by the criteria applied to real recordings. (**a**) Fraction of units classified as place cells (Skaggs spatial information above threshold) and as border-modulated (a third or more of the rate-map mass in the outer ring), by head-direction encoding, pooled across arenas. The composition is stable across encodings. (**b**) Fields per place cell by encoding and arena ($C_1/C_2/C_4$): it rises with both encoding invariance and arena symmetry, from 1.78 to 3.39, the multi-field signature of the fold. Folding rearranges where fields sit and multiplies them without changing the cell-type makeup. Bars are mean ± s.e.m. over networks.

3 Population manifolds of the topology arenas

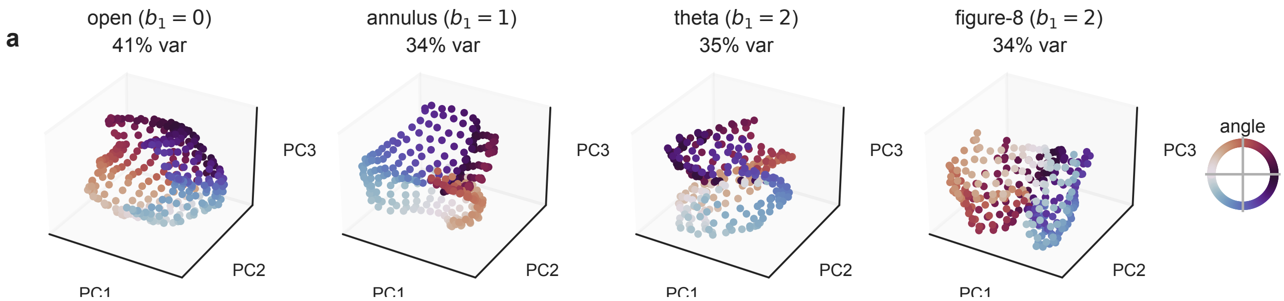

Figure S6. Position-conditioned population manifolds for the four topology arenas, PCA to three dimensions (one point per passable cell, coloured by angle around the arena centre). Position is mapped smoothly in every arena, but the manifolds are high-dimensional: three principal components capture only 34–41% of the variance, and no low-dimensional loop is visible even for the annulus ($b_1 = 1$) or the theta and figure-8 arenas ($b_1 = 2$). This is consistent with the persistent-homology analysis, which does not recover the arenas' loop topology (main text, Limitations): the code is metrically organised well before, and without ever cleanly acquiring, the arena's topology.

4 Diffusion-map geometry of the manifolds

Diffusion maps embed a point cloud through the eigenfunctions of a Markov random walk built on it, so the embedding respects diffusion distance — a density-robust, geometry-aware metric — and the eigenvalue spectrum reads out intrinsic dimension. Both structures survive this nonlinear readout.

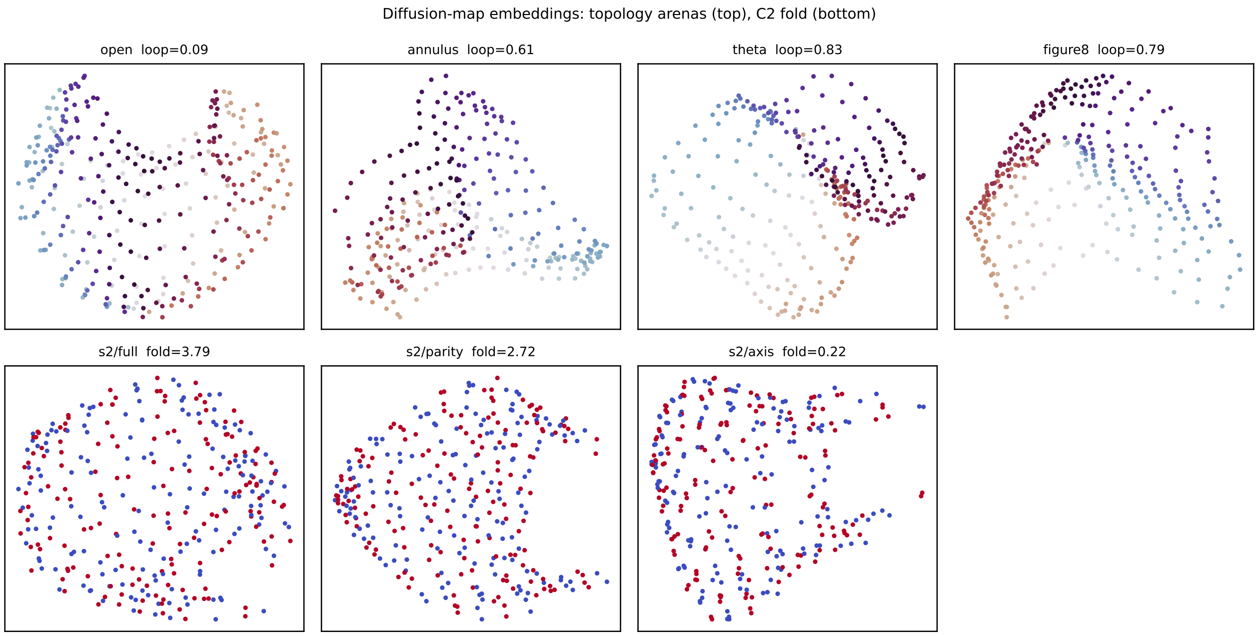

Figure S7. Diffusion-map embeddings. **Top:** topology arenas, coloured by arena angle. The open arena shows no angular loop in the leading diffusion coordinates (loop score 0.09), whereas the annulus, theta and figure-8 arenas do (0.61–0.83); the manifolds nonetheless remain moderately high-dimensional (intrinsic dimension 5–7), consistent with the persistent-homology result that a single loop is recoverable but higher-genus topology is not. **Bottom:** the C_2 fold, coloured by orbit tag. The diffusion distance between a position and its 180° image, in units of the distance to a spatial neighbour, is 3.79 under *full* and 2.72 under *parity* but collapses to 0.22 under *axis*: the fold is a property of the diffusion geometry, not of any linear projection.

5 Initialization and learning speed

The recurrent matrix is not initialized at random alone: the architecture adds a leaky-integrator term $(1 - 1/\tau)\, I$, and a comment in the original code records that a Kaiming initialization was tried and discarded because it "blows up". We varied the initialization along three axes — the identity time constant τ, the gain of the random component, and its structure — and summarised each run by the area under its training-loss curve (Fig. S8). Learning speed ordered monotonically with the amount of random recurrent mixing present at initialization. A pure leak, $W = 0.5\, I$ with no random recurrence, learned fastest; larger random components learned more slowly, and the two Kaiming variants were slowest of all. Neither Kaiming variant diverged: its loss fell monotonically to within 33% of the default by area, with no sign of instability. The blow-up recorded in the original comment belongs to the plain recurrent cell, not to the layer-normalised cell this network uses. This analysis bears on training dynamics rather than on the symmetry results of the main text, and is reported here for completeness.

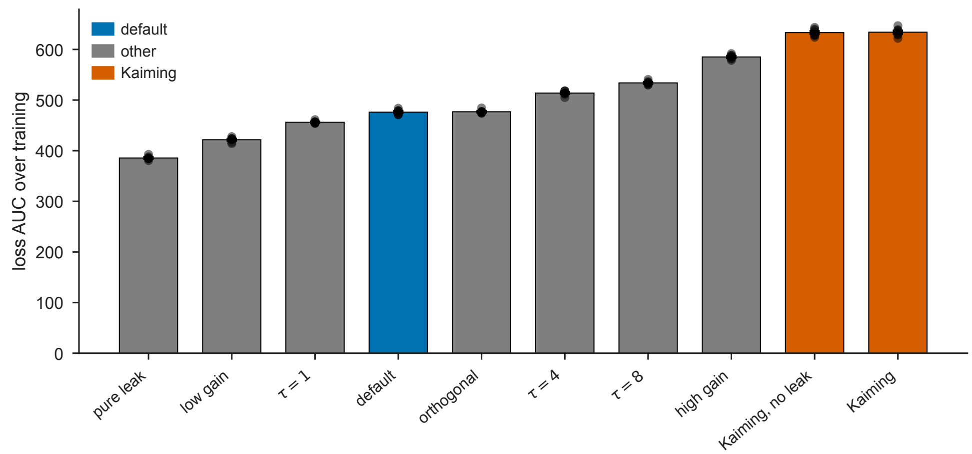

Figure S8. Learning speed by recurrent initialization, measured as area under the training-loss curve (lower is faster), ranked. A pure leak with no random recurrence learns fastest; adding random mixing slows learning monotonically. The abandoned Kaiming initialization is slowest but converges without instability. Dots are individual networks.